

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №2

Выполнил:

Зотеев Максим

БР1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Опираясь на технический дизайн микросервисной архитектуры из ДЗ4, реализовать разделение монолитного бэкенд-приложения сервиса аренды недвижимости на отдельные микросервисы. Каждый сервис должен иметь собственную базу данных (database per service), общаться с другими сервисами по внутренним HTTP-контрактам и быть упакован в Docker

Ход работы

1. Состав сервисов и распределение ответственности

По плану из ДЗ4 монолит распилен на пять компонентов: api-gateway (порт 8080), identity-service (3001), property-service (3002), rental-service (3003) и messaging-service (3004). Identity отвечает за регистрацию, вход и хранение профиля. Property владеет каталогом объектов, фото, удобствами и справочниками типов недвижимости. Rental хранит сделки с денормализованными снапшотами объекта. Messaging — переписку участников сделки и системные уведомления. Api-gateway не хранит данных и занимается только маршрутизацией внешних запросов в нужный сервис.

2. Технологический стек

Все доменные сервисы написаны на Node.js 20 и TypeScript с использованием Express и TypeORM. Это сознательный выбор — стек унаследован от монолита ЛР1, что позволило копировать большую часть бизнес-логики с минимальными правками. Api-gateway также реализован на Express, маршрутизация выполнена через http-proxy-middleware с фильтрацией по пути. Сборка каждого сервиса в отдельный Docker-образ, всё поднимается одной командой docker compose up.

3. Database per service в общем контейнере PostgreSQL

По принципу database-per-service каждый сервис ходит только в свою БД. Для учебного стенда все четыре БД (identity, property, rental, messaging) живут в одном контейнере PostgreSQL 16, что упрощает развёртывание. Создание баз вынесено в SQL-скрипт postgres/init/01-create-databases.sql, который монтируется в /docker-entrypoint-initdb.d и автоматически выполняется при первом старте контейнера. На уровне моделей FK на чужие таблицы запрещены: внешние идентификаторы (owner_id в property, tenant_id и owner_id в rental, sender_id и rental_id в messaging) хранятся как обычные bigint без связей.

4. Снапшоты в сделках аренды

В таблице rentals добавлены поля property_title_snapshot, property_city_snapshot и price_per_month_snapshot. Они заполняются rental-сервисом в момент создания сделки данными, полученными из property-service по внутреннему эндпоинту /internal/properties/:id/rental-context. Это даёт три преимущества: список «мои сделки» строится одним запросом в собственную БД без N+1 HTTP-вызовов; история сделок сохраняет цену и название объекта на момент бронирования даже после правок

арендодателем; сервис способен показать список сделок даже при временной недоступности property-service.

5. Сквозная аутентификация и внутренний токен

JWT выписывает identity-service, а валидируют его все сервисы локально через общий JWT_SECRET. Api-gateway не разбирает токены, а только пробрасывает заголовок Authorization. Для межсервисных вызовов введён отдельный заголовок X-Internal-Service-Token, проверяемый middleware internalAuth каждого сервиса. Маршруты вида /api/v1/internal/* доступны только внутри сети Docker — api-gateway явно отрезает их от внешнего трафика через pathFilter, так что прямой запрос к /api/v1/internal/users/1 снаружи возвращает 404.

6. Межсервисное взаимодействие по HTTP

Синхронные контракты задокументированы в microservices-openapi.yaml из Д34. Rental-service ходит в property-service за rental-context и обновлением availability, а в identity-service — за обогащением ответов профилем пользователей через POST /internal/users/batch-get. Messaging-service ходит в rental-service за участниками сделки (для проверки доступа к чату) и в identity-service за данными отправителей сообщений. Все межсервисные вызовы выполняются через единый клиент internalCall с глобальным таймаутом 5 секунд.

7. Api-gateway и маршрутизация

Шлюз поднят на Express + http-proxy-middleware. Маршруты сопоставлены по префиксам и регуляркам в порядке от более специфичных к более общим: /api/v1/users/me/properties уходит в property-service, /api/v1/users/me/rentals — в rental-service, остальные /api/v1/users/* и /api/v1/auth/* — в identity-service. /api/v1/rentals/:id/messages обрабатывается messaging-сервисом, /api/v1/rentals/* — rental-сервисом, /api/v1/properties/*, /api/v1/property-types, /api/v1/amenities — property-сервисом. /api/v1/internal/* отсекается на уровне pathFilter и не пробрасывается никуда — внутренние эндпоинты доступны только во внутренней docker-сети.

8. Запуск в Docker Compose

docker-compose.yml поднимает шесть сервисов: postgres с healthcheck и смонтированным init-скриптом, rabbitmq с management-плагином для Д35, четыре прикладных сервиса и api-gateway. Зависимости между контейнерами выражены через depends_on с условием service_healthy для БД и брокера, что предотвращает старт прикладных сервисов до готовности инфраструктуры. Каждый сервис собирается отдельным многослойным Dockerfile (npm install → копирование исходников → tsc → запуск dist/index.js). Конфигурация чувствительных значений (JWT_SECRET, INTERNAL_SERVICE_TOKEN) вынесена в .env.

9. Тестирование через Postman

Для приёмки подготовлена коллекция postman/rental-api.lab2.postman_collection.json из 62 тестов и 127 assert'ов, покрывающая регистрацию и валидацию, права арендатора и

арендодателя, CRUD объектов и фотографий, поиск с фильтрами и сортировками, полный жизненный цикл сделки, поведение снапшотов (после изменения title и цены у объекта сделка возвращает прежние значения), переходы статусов и их идемпотентность (повторный complete/cancel → 409), а также изоляцию внутренних эндпоинтов от gateway. Прогон через newman run завершается со 100% успехом

Заключение

В рамках ЛР2 монолитное приложение сервиса аренды недвижимости разделено на пять микросервисов по архитектуре, спроектированной в Д34. Реализованы собственные базы данных для каждого сервиса (изолированные внутри одного контейнера PostgreSQL), синхронное межсервисное взаимодействие по HTTP с внутренним служебным токеном, денормализация через снапшоты в сделках и api-gateway с маршрутизацией по префиксам. Корректность подтверждена прогоном Postman-коллекции из 62 сценариев, включая негативные. Полученная архитектура готова к расширению асинхронной интеграцией через RabbitMQ, которая реализована в рамках Д35